

Patrón de Diseño Proxy

Un patrón de diseño estructural que permite proporcionar un sustituto o marcador de posición para otro objeto, controlando el acceso al objeto original.



Agenda

01

Propósito

Qué es el patrón Proxy y para qué sirve

03

Analogía y Estructura

Ejemplo del mundo real y diagrama UML

05

Aplicabilidad

Tipos de proxy y cuándo usarlos

02

Problema y Solución

El problema que resuelve y cómo lo hace

04

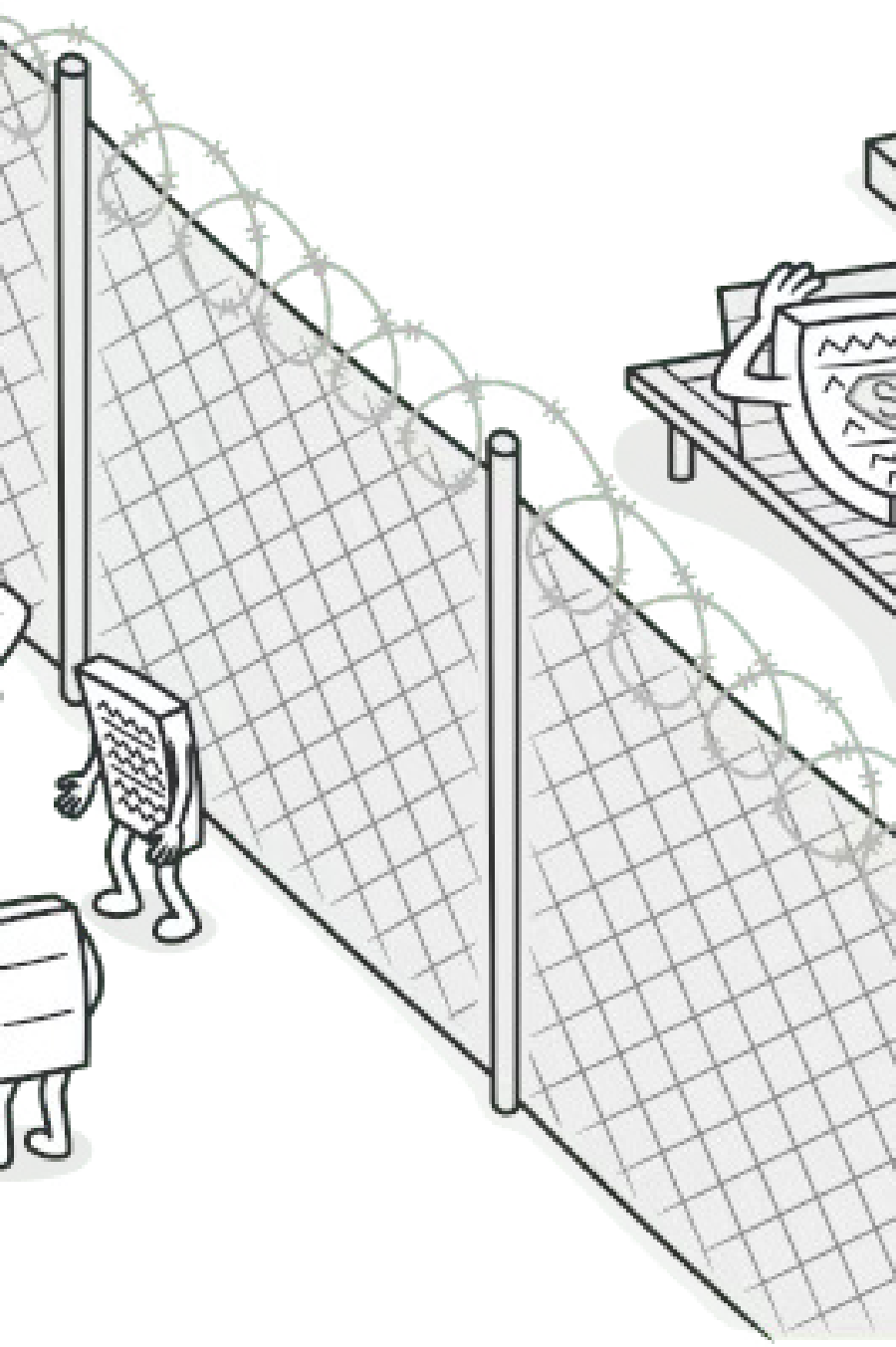
Pseudocódigo

Implementación completa con ejemplo de YouTube

06

Implementación, Pros/Contras y Relaciones

Guía práctica y comparación con otros patrones



CAPÍTULO 1

Propósito del Patrón Proxy

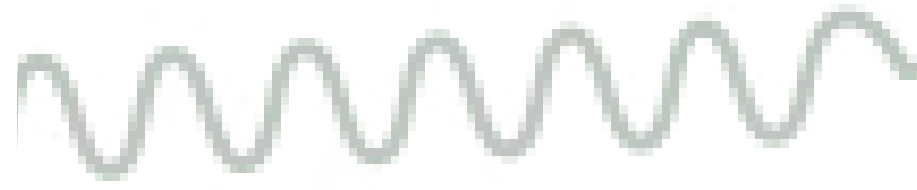
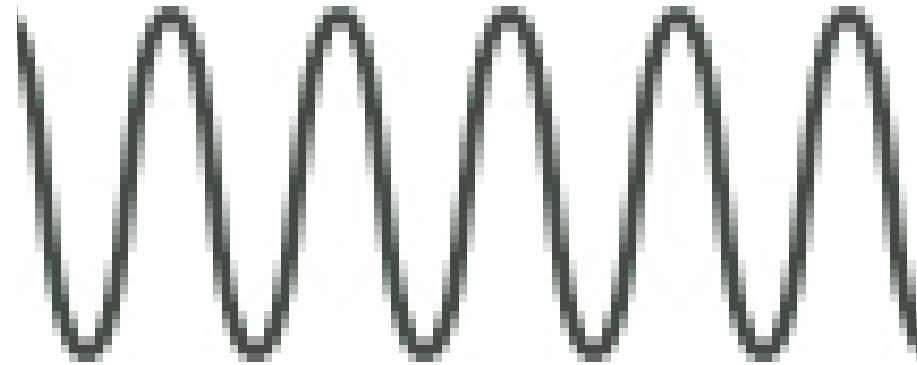
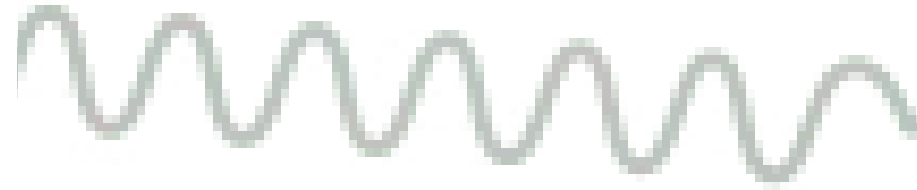
Proxy es un **patrón de diseño estructural** que te permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo **antes o después** de que la solicitud llegue al objeto original.

CAPÍTULO 2

El Problema

¿Por qué querrías controlar el acceso a un objeto? Imagina que tienes un **objeto enorme** que consume una gran cantidad de recursos del sistema. Lo necesitas de vez en cuando, pero no siempre.

Las consultas a las bases de datos pueden ser muy lentas.



El Problema: Código Duplicado

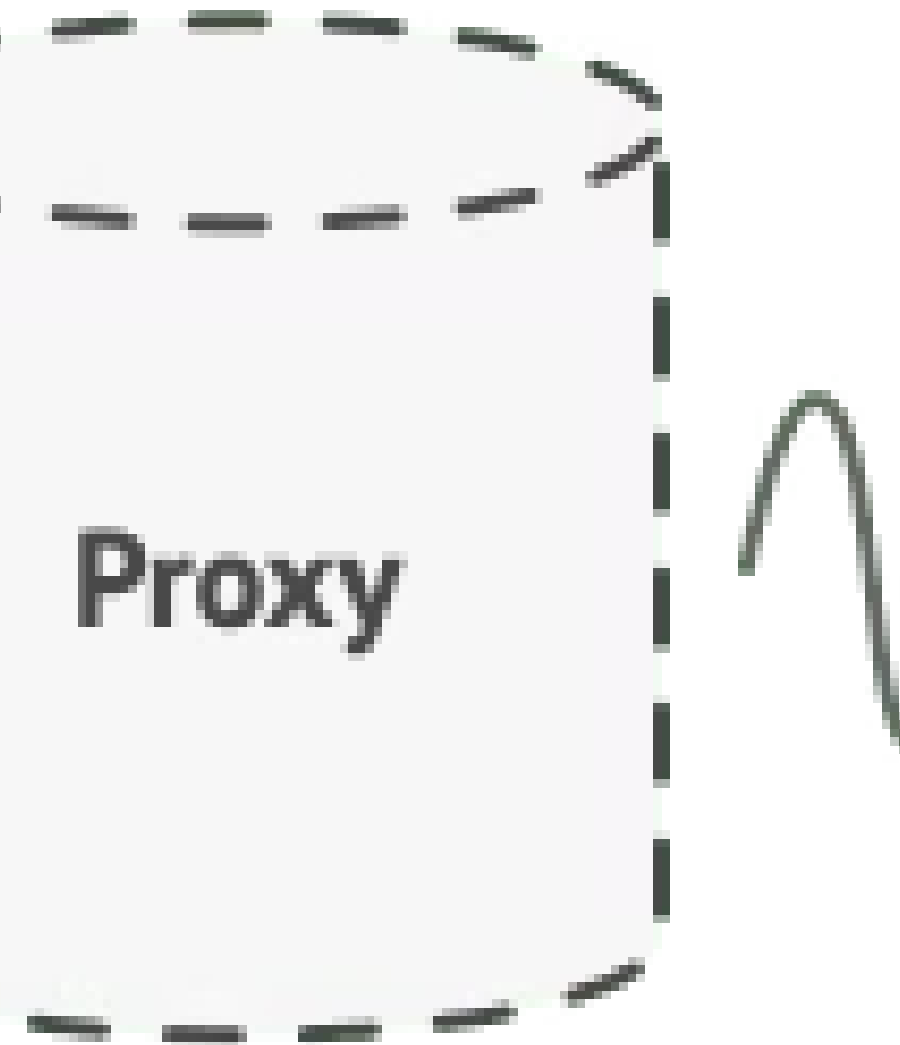
Puedes llevar a cabo una **implementación diferida**, es decir, crear este objeto sólo cuando sea realmente necesario. Todos los clientes del objeto tendrán que ejecutar algún código de inicialización diferida.

Lamentablemente, esto seguramente generará una **gran cantidad de código duplicado**.

En un mundo ideal, querríamos meter este código directamente dentro de la clase de nuestro objeto, pero eso no siempre es posible. Por ejemplo, la clase puede ser parte de una **biblioteca cerrada de un tercero**.

Desafíos

- Código de inicialización diferida duplicado en cada cliente
- No siempre se puede modificar la clase original
- Bibliotecas de terceros son inaccesibles



CAPÍTULO 3

La Solución

El patrón Proxy sugiere que crees una **nueva clase proxy** con la misma interfaz que un objeto de servicio original. Después actualizas tu aplicación para que pase el objeto proxy a todos los clientes del objeto original.

Al recibir una solicitud de un cliente, el proxy crea un **objeto de servicio real** y le delega todo el trabajo.

El proxy se camufla como objeto de la base de datos. Puede gestionar la inicialización diferida y el caché de resultados sin que el cliente o el objeto real de la base de datos lo sepan.

Ventaja Clave del Proxy

Ejecutar lógica adicional

Si necesitas ejecutar algo antes o después de la lógica primaria de la clase, el proxy te permite hacerlo **sin cambiar esa clase**.

Misma interfaz

Ya que el proxy implementa la misma interfaz que la clase original, puede pasarse a **cualquier cliente** que espere un objeto de servicio real.

Transparencia

El cliente no necesita saber si trabaja con el objeto real o con el proxy. La sustitución es **completamente transparente**.



CAPÍTULO 4

Analogía en el Mundo Real

Las tarjetas de crédito pueden utilizarse para realizar pagos tanto como el efectivo.

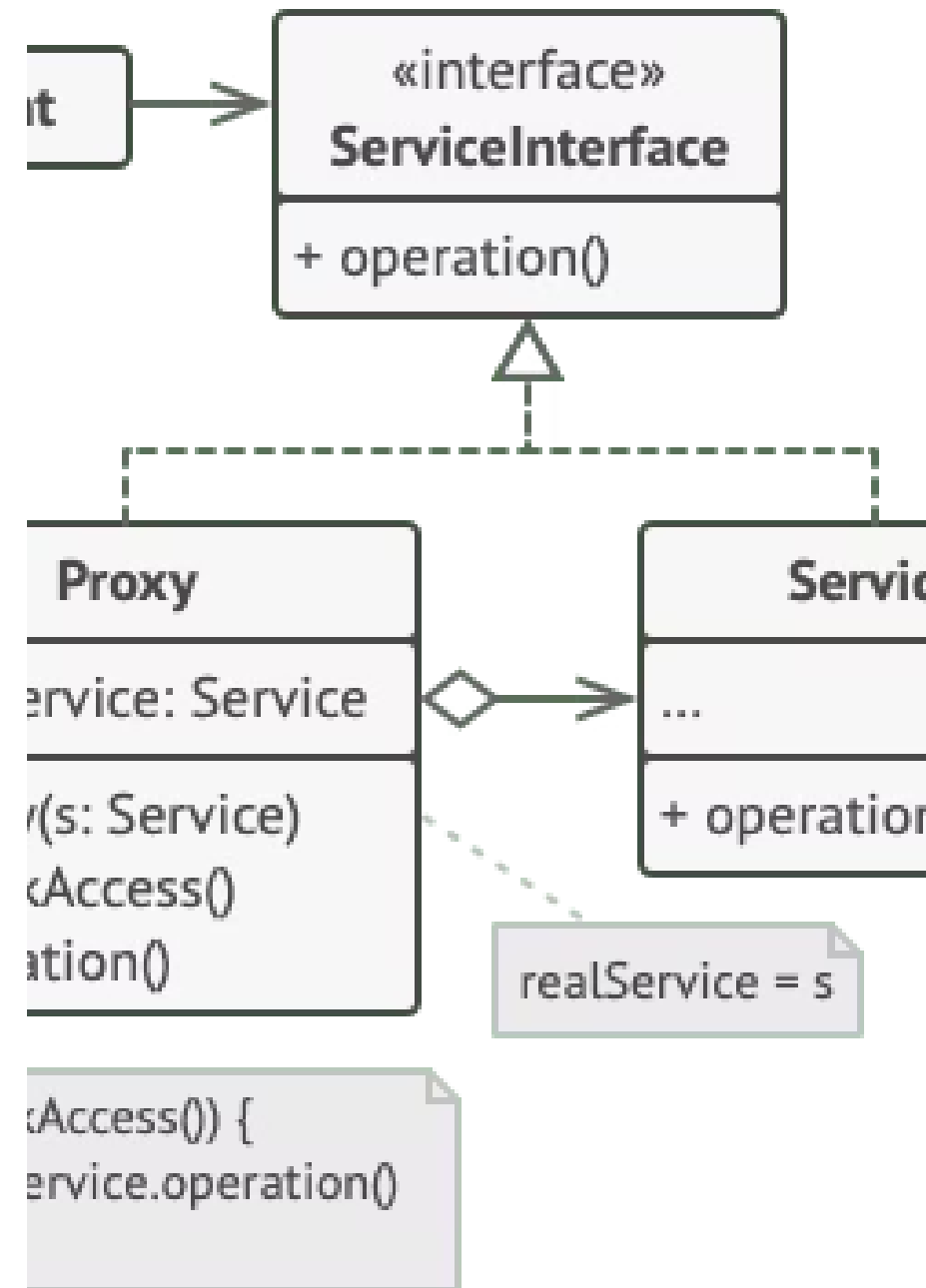
La Tarjeta de Crédito como Proxy

Una **tarjeta de crédito** es un proxy de una cuenta bancaria, que, a su vez, es un proxy de un manojito de billetes. Ambos implementan la misma interfaz, por lo que pueden utilizarse para realizar un pago.



El consumidor no necesita llevar efectivo encima. El dueño de la tienda recibe los ingresos electrónicamente sin el riesgo de perder el depósito o sufrir un robo de camino al banco.

Estructura del Patrón Proxy



Componentes de la Estructura

1

Interfaz de Servicio

Declara la interfaz del Servicio. El proxy debe seguir esta interfaz para poder **camuflarse** como objeto de servicio.

2

Servicio

Es una clase que proporciona una **lógica de negocio útil**.

3

Clase Proxy

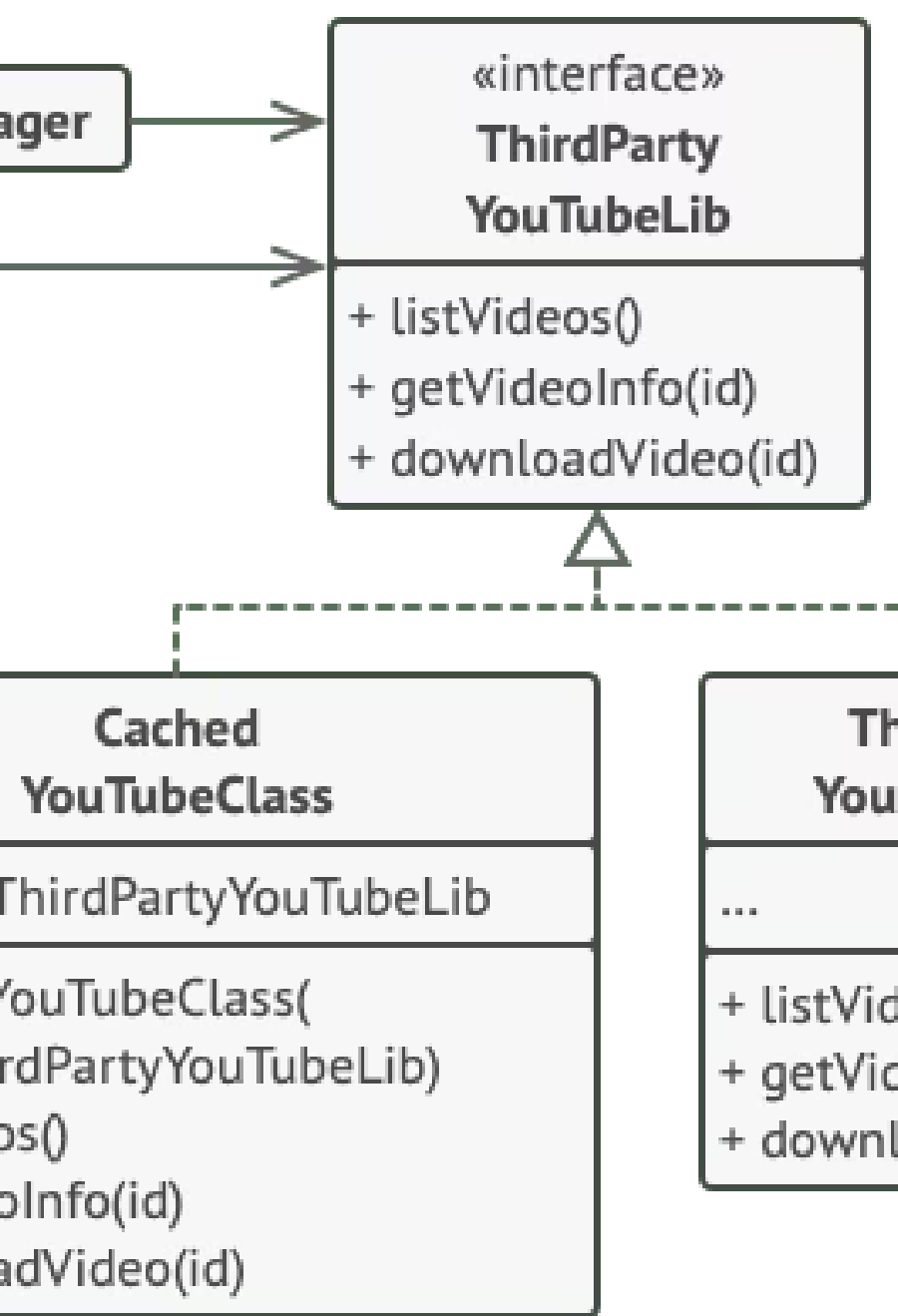
Tiene un campo de referencia que apunta a un objeto de servicio. Tras su procesamiento (inicialización diferida, registro, control de acceso, caché), **pasa la solicitud** al objeto de servicio.

4

Cliente

Debe funcionar con servicios y proxies a través de la **misma interfaz**. Así puedes pasar un proxy a cualquier código que espere un objeto de servicio.

Normalmente los proxies gestionan el **ciclo de vida completo** de sus objetos de servicio.



CAPÍTULO 6

Pseudocódigo: Ejemplo con YouTube

Este ejemplo ilustra cómo el patrón Proxy puede ayudar a introducir la **inicialización diferida** y el **almacenamiento en caché** a una biblioteca de integración de YouTube de un tercero.

Resultados del almacenamiento en caché de un servicio con un proxy.

El Problema con la Biblioteca

La biblioteca nos proporciona la clase de descarga de videos. Sin embargo, es **muy ineficiente**. Si la aplicación cliente solicita el mismo video muchas veces, la biblioteca lo descarga una y otra vez, en lugar de guardarlo en caché y reutilizar el primer archivo descargado.

La clase proxy implementa la **misma interfaz** que el descargador original y le delega todo el trabajo. No obstante, mantiene un seguimiento de los archivos descargados y devuelve los **resultados en caché** cuando la aplicación solicita el mismo video varias veces.

Solución Proxy

- Misma interfaz que el original
- Delega el trabajo real
- Mantiene caché de descargas
- Reutiliza archivos existentes

Interfaz del Servicio Remoto

Definición de la interfaz `ThirdPartyYouTubeLib` que tanto el servicio real como el proxy implementarán:

```
// La interfaz de un servicio remoto.  
interface ThirdPartyYouTubeLib is  
    method listVideos()  
    method getVideoInfo(id)  
    method downloadVideo(id)
```

Implementación Concreta del Servicio

La clase `ThirdPartyYouTubeClass` implementa el conector real. La velocidad depende de la conexión a internet y se ralentiza con muchas solicitudes simultáneas.

```
// La implementación concreta de un conector de servicio. Los
// métodos de esta clase pueden solicitar información a YouTube.
// La velocidad de la solicitud depende de la conexión a
// internet del usuario y de YouTube. La aplicación se
// ralentizará si se lanzan muchas solicitudes al mismo tiempo,
// incluso aunque todas soliciten la misma información.
class ThirdPartyYouTubeClass implements ThirdPartyYouTubeLib is
method listVideos() is
// Envía una solicitud API a YouTube.
method getVideoInfo(id) is
// Obtiene metadatos de algún video.
method downloadVideo(id) is
// Descarga un archivo de video de YouTube.
```

La Clase Proxy con Caché (Parte 1)

Para ahorrar ancho de banda, guardamos en caché los resultados. No podemos colocar este código dentro de la clase de servicio (puede ser de un tercero o definida como `final`), por eso usamos una clase proxy.

```
// Para ahorrar ancho de banda, podemos guardar en caché
// resultados de la solicitud durante algún tiempo, pero se
// puede colocar este código directamente dentro de la clase de
// servicio. Por ejemplo, puede haberse proporcionado como parte
// de la biblioteca de un tercero y/o definido como `final`. Por
// eso colocamos el código de almacenamiento en caché dentro de
// una nueva clase proxy que implementa la misma interfaz que la
// clase servicio. Delega al objeto de servicio únicamente
// cuando deben enviarse las solicitudes reales.
```

```
class CachedYouTubeClass implements ThirdPartyYouTubeLib is
private field service: ThirdPartyYouTubeLib
private field listCache, videoCache
field needReset
```

```
constructor CachedYouTubeClass(service: ThirdPartyYouTubeLib) is
this.service = service
```


La Clase Proxy con Caché (Parte 2)

Los métodos del proxy verifican si existe caché antes de delegar al servicio real:

```
method listVideos() is  
if (listCache == null || needReset)  
listCache = service.listVideos()  
return listCache
```

```
method getVideoInfo(id) is  
if (videoCache == null || needReset)  
videoCache = service.getVideoInfo(id)  
return videoCache
```

```
method downloadVideo(id) is  
if (!downloadExists(id) || needReset)  
service.downloadVideo(id)
```

La Clase GUI (YouTubeManager)

La clase GUI permanece **sin cambios** siempre y cuando trabaje con el objeto de servicio a través de una interfaz. Podemos pasar un objeto proxy en lugar de un objeto real, ya que ambos implementan la misma interfaz.

```
// La clase GUI, que solía trabajar directamente con un objeto
// de servicio, permanece sin cambios siempre y cuando trabaje
// con el objeto de servicio a través de una interfaz. Podemos
// pasar sin riesgo un objeto proxy en lugar de un objeto de
// servicio real, ya que ambos implementan la misma interfaz.
```

```
class YouTubeManager is
```

```
protected field service: ThirdPartyYouTubeLib
```

```
constructor YouTubeManager(service: ThirdPartyYouTubeLib) is
```

```
this.service = service
```

```
method renderVideoPage(id) is
```

```
info = service.getVideoInfo(id)
```

```
// Representa la página del video.
```

```
method renderListPanel() is
```

```
list = service.listVideos()
```

```
// Representa la lista de miniaturas de los videos.
```

```
method reactOnUserInput() is
```

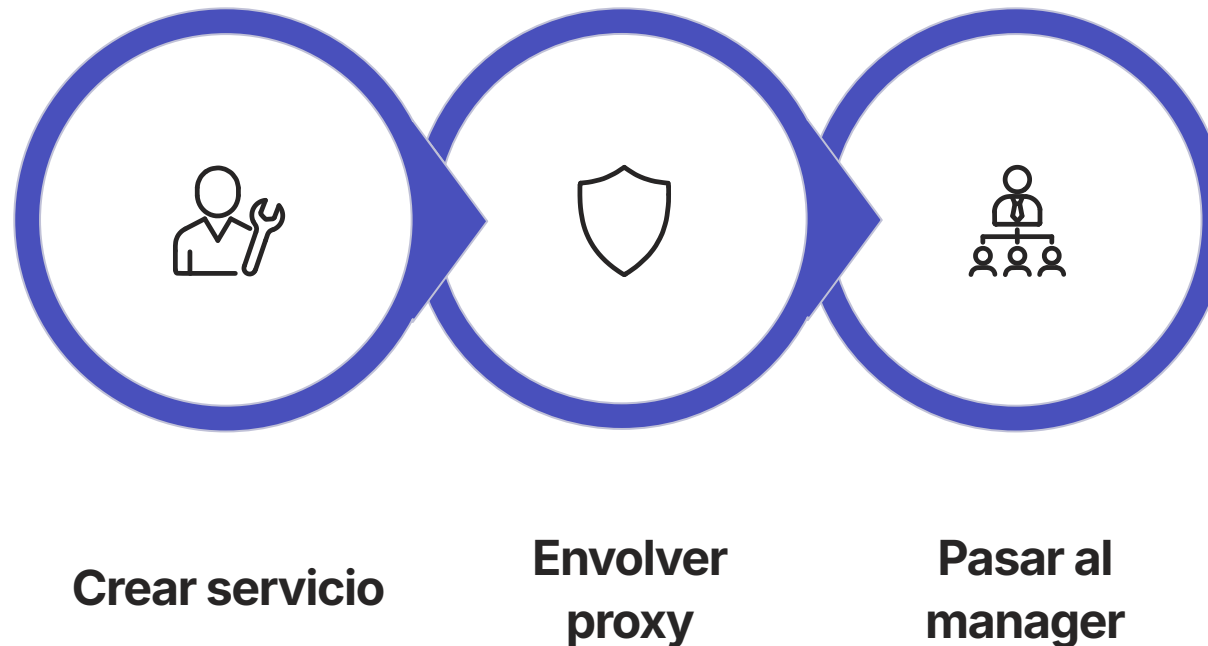
```
renderVideoPage()
```

```
renderListPanel()
```

Configuración de la Aplicación

La aplicación puede configurar proxies sobre la marcha. Aquí se crea el servicio real, se envuelve con el proxy de caché y se pasa al manager:

```
// La aplicación puede configurar proxies sobre la marcha.  
class Application is  
  method init() is  
    aYouTubeService = new ThirdPartyYouTubeClass()  
    aYouTubeProxy = new CachedYouTubeClass(aYouTubeService)  
    manager = new YouTubeManager(aYouTubeProxy)  
    manager.reactOnUserInput()
```



El manager nunca sabe si trabaja con el servicio real o con el proxy.

Aplicabilidad: Tipos de Proxy

Hay decenas de formas de utilizar el patrón Proxy. Repasemos los usos más populares.

1

Proxy Virtual

Inicialización diferida. Cuando tienes un objeto de servicio muy pesado que utiliza muchos recursos al estar siempre funcionando, aunque solo lo necesites de vez en cuando. Puedes retrasar la inicialización a un momento en que sea realmente necesario.

2

Proxy de Protección

Control de acceso. Cuando quieres que únicamente clientes específicos sean capaces de utilizar el objeto de servicio. El proxy pasa la solicitud solo si las credenciales del cliente cumplen ciertos criterios.

3

Proxy Remoto

Ejecución local de un servicio remoto. Cuando el objeto de servicio se ubica en un servidor remoto. El proxy pasa la solicitud del cliente por la red, gestionando todos los detalles de trabajar con la red.

Más Tipos de Proxy

1

Proxy de Registro

Solicitudes de registro. Cuando quieres mantener un historial de solicitudes al objeto de servicio. El proxy puede registrar cada solicitud antes de pasarla al servicio.

2

Proxy de Caché

Resultados de solicitudes en caché. Cuando necesitas guardar en caché resultados de solicitudes de clientes y gestionar el ciclo de vida de ese caché. El proxy puede utilizar los parámetros de las solicitudes como claves de caché.

3

Referencia Inteligente

Cuando debes ser capaz de **desechar un objeto pesado** una vez que no haya clientes que lo utilicen. El proxy rastrea los clientes que obtuvieron una referencia y, si la lista se vacía, desecha el objeto y libera recursos. También puede rastrear si el cliente ha modificado el objeto para reutilizar objetos sin cambios.

Cómo Implementarlo

1 Crear la interfaz de servicio

Si no hay una interfaz preexistente, crea una para que proxy y servicio sean intercambiables. Si no es posible, convierte el proxy en **subclase** de la clase servicio para heredar su interfaz.

2 Crear la clase proxy

Debe tener un campo para almacenar una **referencia al servicio**. Normalmente los proxies crean y gestionan el ciclo de vida completo de sus servicios.

3 Implementar los métodos

Según sus propósitos. En la mayoría de los casos, después de hacer cierta labor, el proxy debería **delegar el trabajo** a un objeto de servicio.

4 Método de creación

Considera introducir un método que decida si el cliente obtiene un proxy o un servicio real. Puede ser un **método estático** o un método de fábrica.

5 Inicialización diferida

Considera implementar la **inicialización diferida** para el objeto de servicio.

Pros y Contras

✓ Ventajas

- Puedes controlar el objeto de servicio sin que los clientes lo sepan.
- Puedes gestionar el ciclo de vida del objeto de servicio cuando a los clientes no les importa.
- El proxy funciona incluso si el objeto de servicio no está listo o no está disponible.
- **Principio de abierto/cerrado.** Puedes introducir nuevos proxies sin cambiar el servicio o los clientes.

✗ Desventajas

- El código puede complicarse ya que debes introducir gran cantidad de clases nuevas.
- La respuesta del servicio puede retrasarse.

Relaciones con Otros Patrones

Adapter vs Proxy vs Decorator

Con **Adapter** se accede a un objeto existente a través de una interfaz diferente. Con **Proxy**, la interfaz sigue siendo la misma. Con **Decorator** se accede al objeto a través de una interfaz mejorada.

Facade vs Proxy

Facade es similar a Proxy en que ambos pueden almacenar temporalmente una entidad compleja e inicializarla por su cuenta. Al contrario que Facade, **Proxy tiene la misma interfaz** que su objeto de servicio, lo que hace que sean intercambiables.

Decorator vs Proxy

Ambos tienen estructuras similares y se basan en el **principio de composición**. La diferencia: un Proxy gestiona el ciclo de vida de su objeto de servicio por su cuenta, mientras que la composición de los **Decoradores siempre está controlada por el cliente**.

Resumen Final

El patrón **Proxy** es una herramienta poderosa para controlar el acceso a objetos, gestionar recursos pesados y añadir funcionalidad sin modificar las clases originales. Desde la inicialización diferida hasta el almacenamiento en caché, este patrón ofrece soluciones elegantes para problemas comunes en el diseño de software.

Misma interfaz

Control de acceso

**Transparente al
cliente**

Principio abierto/cerrado

